GENERATIVE AI BOOTCAMP · POST-SESSION NOTES

Vibe Coding. The Foundations.

A structured recap of the 60-minute primer — plus a practice plan, glossary, and reflection prompts to take you from concept to first shipped build.

SESSION 01

01

MODULE

Foundations → Build

INSIDE THIS DOCUMENT

What you'll take away from these notes.

Everything covered in the session, reorganised for reference — plus the extras you need to actually practise between now and the next session.

SECTION 01 · THE MOMENT

When vibe coding became a real way to build.

On **February 2, 2025**, Andrej Karpathy — co-founder of OpenAI — posted a single tweet describing a new way of coding: you stop writing the code and start describing what you want. He called it **vibe coding**.

Why that tweet mattered

It named something that had quietly become possible. For the first time, you could describe a feature in plain language and the model would plan, write, and test the code well enough for the result to work. The term stuck because it captured a real shift — not in syntax, but in **who gets to build**.

IN KARPATHY'S OWN FRAMING

You "fully give in to the vibes, embrace exponentials, and forget that the code even exists." You describe what you want. The model produces it. You read the behaviour, not the source.

The scale of adoption

From roughly zero daily use in 2022 to a near-majority of developers in 2025.

2022

~0%

AI coding tools were virtually absent from daily developer workflow.

2024

20%

Adoption accelerates as tools mature and models improve.

2025

44%

Crossed 44% daily use — a majority is within reach.

Takeaway. This is no longer a fringe workflow. The people who commit to it now are the ones who will ship before everyone else catches on.

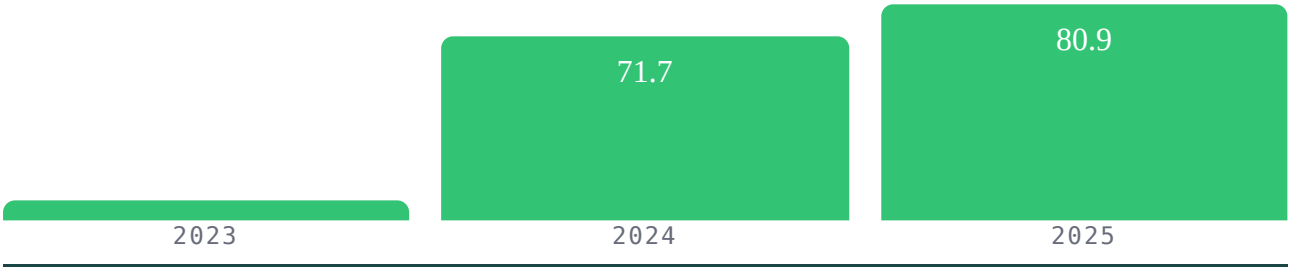
SECTION 02 · WHY NOW

The capability jump that made it possible.

Vibe coding became viable because AI models crossed a real performance threshold on non-trivial coding tasks.

SWE-Bench — the real coding test

SWE-Bench gives the model a real bug from a real open-source project and checks whether the model can fix it. It is not autocomplete. It is engineering.



SWE-Bench Verified score (%), by year. A 4.4% score means the model could barely fix anything real. An 80.9% score means it closes out most bugs without hand-holding — the difference between a toy and a tool.

From autocomplete to agent

BEFORE 2025

You were the driver.

AI suggested the next line. You still wrote every function, debugged every error, and made every decision.

2025 AND BEYOND

AI became an agent.

You describe the feature. It plans, writes, tests, fixes its own bugs, and hands the result back.

You stopped being a coder. You became a **director**.

SECTION 03 · THE CORE SHIFT

Building software is no longer a technical skill.

It is a **communication skill**. If you can write a clear brief, you can build an app. The constraint has moved from *knowing how to code* to *knowing what to build and how to describe it*.

What actually changed

Every app you use — Zomato, PhonePe, Cred, Swiggy — was built by a team of engineers over months. For the first time, you can build something in that same category without being one of them. The craft has not disappeared; it has shifted.

WHAT USED TO BE HARD

- Memorising syntax and frameworks
- Writing boilerplate by hand
- Stitching libraries together
- Configuring servers and deployment

WHAT MATTERS NOW

- Clarity on the problem you're solving
- Specifying features in plain language
- Reading output and catching what's wrong
- Iterating fast instead of hoping for perfection

THE MENTAL MODEL TO KEEP

Treat the AI like a fast junior engineer.

Junior engineers build exactly what you describe. If your brief is vague, the output is vague. If your brief is specific, the output is usable. Your leverage lives in the brief.

SECTION 04 · THE THREE LAYERS

Every app is these three things working together.

Hold this model in your head for the rest of your building career. Whatever you use — Instagram, Uber, a bank app, a bootcamp dashboard — is a combination of these three layers.

01

FRONTEND

What you see and tap.

The images, the buttons, the colours, the search bar, the menus. When someone calls an app "clean," they are talking about the frontend. It is the face of the app.

02

BACKEND

What happens after you tap.

You tap "Order" — the backend finds the restaurant, checks if it's open, assigns a rider, calculates delivery time. It runs on servers you cannot see. It is the brain of the app: invisible, essential, always running.

03

DATABASE

What the app remembers.

Close any app and open it again tomorrow — your past data is still there. That memory lives inside a database: a large, organised table of everything the app needs to remember.

How this helps you as a vibe coder. When a build breaks, knowing which layer is failing tells you what to prompt next. "The button doesn't appear" is a frontend issue. "The order isn't being saved" is a database issue. "The price isn't calculating" is a backend issue.

SECTION 05 · SUPPORTING CONCEPTS

Three more ideas that show up in every build.

You do not need to implement these yourself. You need to recognise them when the AI uses them, and know what to ask for when a feature requires one of them.

CONCEPT 01

API · Application Programming Interface

APIs are how one piece of software talks to another. Your weather app does not own weather data — it *calls an API* that does. Your payment screen does not own your bank account — it *calls an API* at Razorpay or Stripe.

The waiter analogy. You (the frontend) tell the waiter (the API) what you want. The waiter goes to the kitchen (the backend / another service), brings back the result, and serves it to you. You never step into the kitchen — you just describe the order clearly.

CONCEPT 02

Authentication · Who's allowed in

Authentication covers login, signup, forgot-password, Google sign-in, and — most importantly — making sure one user cannot see another user's data. Every app with accounts needs this.

In vibe coding, tools like **Supabase Auth** and **Clerk** handle this for you. You don't write the login logic — you plug it in.

CONCEPT 03

Hosting · Where the app lives on the internet

Your code has to live somewhere for anyone to use it. That place is called a host.

2015

You rented a server and configured it yourself. Days of setup before anyone could visit your URL.

2026

Platforms like **Vercel** and **Netlify** do it in one click. **Lovable** and **Bolt** do it automatically.

SECTION 06 · DEFINITION & LOOP

Vibe coding, in one line.

You describe what you want. AI builds it.

"Build me a landing page for a yoga studio with a booking form." The AI handles the frontend, the backend, the database, and the hosting. You refine. It updates. You ship.

The loop — the actual workflow

Beginners write one massive prompt and expect perfection. People who ship run this loop twenty times in a row.

STEP 01 · DESCRIBE

Describe.

Write a clear prompt explaining what you want to build or change. Be specific: user, action, outcome.

STEP 02 · GENERATE

Generate.

The AI writes the code, creates the files, and assembles the feature. You watch, you don't type.

STEP 03 · TEST

Test.

Click through the result. Does it work? Does it look right? Does the data save?

STEP 04 · REFINE

Refine.

Describe what needs to change — in plain language. Go again. Small diffs beat big rewrites.

Rule of thumb. If a prompt is more than three sentences long, break it into two prompts. The loop rewards small, clear moves.

SECTION 07 · THE 80/20 REALITY

The first 80% is easy. The last 20% is where most people stop.

This is the single most important idea for anyone starting out. Knowing where the difficulty lives stops you from quitting when the hard part arrives — because you'll know it was always going to arrive.

FIRST 80%

A working prototype takes an afternoon.

A landing page, a dashboard, a simple form — all achievable in hours with Lovable or Bolt. This is the magical part, and it's real.

Prototype | Demo | Internal tool

LAST 20%

A production app takes months.

Payments, real users, security, edge cases, performance. This is where most people stop — and where the real progression begins.

Payments | Auth | Scale

Why the gap exists

The first 80% is pattern-matching — the AI has seen thousands of landing pages, forms, and dashboards. The last 20% is about **your specific users**, **your specific data**, and **edge cases no one has seen before**. That work needs more control, cleaner prompts, and a tool that lets you touch the code.

The good news. You don't need to cross the last 20% on day one. Most learners ship useful things inside the first 80% for weeks before they need more power. The progression exists exactly for when you're ready.

SECTION 08 · THE TOOL PROGRESSION

The order of learning.

Do not skip stages. The prompting skills you build at Stage 1 are the exact skills you'll need at Stage 2. Stage 3 only makes sense once Stage 2 is second nature.

01

STAGE 01 · START HERE

Prompt-first builders.

You describe. The platform builds, hosts, and deploys. No code to touch, no environment to configure. Your job is to learn to prompt clearly and to get used to the describe → test → refine loop.

Lovable · Bolt · Replit · v0

Graduate when: you hit the limits of what the platform can do, and you start wanting to edit specific files by hand.

02

STAGE 02 · OWN THE CODE

AI-native IDEs.

The code lives on your machine. The AI still writes most of it, but you can jump in, rename a file, fix a single line, plug in external services, and ship more complex features. This is where real products get built.

Cursor · Claude Code · Windsurf

Graduate when: a single AI session isn't enough — you want multiple agents working different parts of the codebase in parallel.

03

STAGE 03 · DIRECT A TEAM

Multi-agent setups.

Run several AI agents in parallel — one on the frontend, one on the backend, one writing tests. You stop writing code entirely and start managing a team of AI coders.

Antigravity · Claude Code (sub-agents) · Orchestrator patterns

One rule above all. Spend more time at Stage 1 than feels comfortable. The habits formed there — clear briefs, small diffs, fast iteration — are the same habits that make Stage 2 and Stage 3 work.

SECTION 09 · NEXT STEPS

Your 7-day practice plan.

The fastest way to internalise the last hour is to ship something small by the end of the week. Do not wait to feel ready. Use this plan as your scaffold.

---

01 **Day 1 · Set up one Stage-1 tool.**
Pick **one** — Lovable, Bolt, or Replit. Create an account. Open an empty project. Resist the urge to try all three.

02 **Day 2 · Rebuild something that exists.**
Pick a small page you already use — a pricing page, a contact form, a menu. Describe it in plain English and let the tool rebuild it. Notice what's off. Refine.

03 **Day 3 · Practise the loop deliberately.**
Take yesterday's build. Make **ten** small changes — one prompt at a time. Change colours, copy, layout, add a field. Short prompts, fast iteration.

04 **Day 4 · Add a database.**
Build a page that remembers something — a simple form that saves submissions to a table. This is the first moment the "three layers" become real.

05 **Day 5 · Add authentication.**
Put a login in front of your form. Try Google sign-in. Confirm that logged-in users only see their own data. You've now touched every layer.

06 **Day 6 · Ship it.**
Publish to a live URL. Share the link with one person and ask them to use it. Fix what they find confusing — in one short prompt at a time.

07 **Day 7 · Reflect & plan next build.**
Write down what worked, what broke, and what you'd want to build next. Come to the next session with the link and your notes.

If you have only one hour a day, that's enough. The loop rewards consistency over marathons. A small build shipped beats a big build abandoned.

SECTION 10 · GLOSSARY

Every term from the session, in one place.

Flip to this page any time a word in a tutorial or prompt feels unfamiliar.

Vibe coding	A way of building software where you describe what you want in plain language and the AI writes, assembles, and (often) deploys the code.
Frontend	The part of the app the user sees and interacts with — screens, buttons, colours, forms.
Backend	The server-side logic that runs invisibly — calculations, business rules, talking to the database.
Database	The organised storage layer where the app remembers data between sessions (users, orders, posts).
API	Application Programming Interface — a contract that lets one piece of software request data or actions from another.
Authentication	The system that controls who can log in and what data they're allowed to see.
Hosting	The service that puts your app on a live URL so other people can use it from their browser or phone.
Prompt	The instruction you give the AI. Quality of output scales directly with quality of prompt.
The Loop	Describe → Generate → Test → Refine. The four-step cycle that produces anything real.
SWE-Bench	A benchmark that measures how well an AI model can fix real bugs in real open-source projects.
Agent	An AI that plans, executes, checks, and corrects its own work across multiple steps — as opposed to one-shot suggestions.
Production	An app that's live and being used by real people, not a demo on your laptop.

SECTION 11 · TOOLS & LINKS

Your starter stack.

A curated list by stage. Start with one tool in Stage 1. Only move on once you've shipped two or three small builds.

Stage 1 — Prompt-first builders

TOOL	BEST FOR	URL
Lovable	Full-stack web apps from a single prompt	lovable.dev
Bolt	Fast web prototypes with in-browser preview	bolt.new
Replit	Quick apps with a built-in code editor	replit.com
v0	Beautiful frontend-only UI from a description	v0.app

Stage 2 — AI-native IDEs

TOOL	BEST FOR	URL
Cursor	VS-Code-style editor with deep AI integration	cursor.com
Claude Code	Terminal-based agentic coding with Claude	claude.com/claude-code
Windsurf	Agentic IDE focused on flow and long tasks	windsurf.com

Backend, database & hosting — plug these in

TOOL	USE IT FOR	URL
Supabase	Database + authentication in one	supabase.com
Clerk	Drop-in user authentication	clerk.com
Vercel	One-click hosting for frontend apps	vercel.com
Netlify	Hosting + forms + serverless functions	netlify.com

Before you open a tool, write your idea as one paragraph in plain English. The paragraph is your first prompt. If the paragraph is fuzzy, the build will be fuzzy.

SECTION 12 · REFLECTION & SELF-ASSESSMENT

Lock it in before you close this document.

Five prompts. Answer them on paper, on your phone, or inside the tool you'll build with. The act of writing — not reading — is what makes this session stick.

01 · In one sentence — what is vibe coding, as you'd explain it to a friend?

02 · Which of the three layers (frontend, backend, database) feels least clear to you right now, and why?

03 · What is the smallest useful thing you could build and ship in the next 7 days?

04 · Which Stage-1 tool will you commit to, and what's your first prompt going to be?

05 · What specifically stops you from starting today? Name it — then decide if it's a real blocker or a feeling.

EVERYTHING SO FAR HAS BEEN THE PREP.

The rest is the practice. Let's build.

Open one tool. Write one paragraph. Run the loop. See you in the next session with a live link in hand.